FORWARD DEPLOYED ENGINEER · PRE-ENTREGA 2

Motor LLM base y validación estructurada

Nicolás Suárez · Caso: normalización de medidas y peso de altas de catálogo (PE1) · Repo: `github.com/<usuario>/fde-medidas-catalogo` (tag `pe2`) · Modelo: `gemini-3.1-flash-lite`

El motor recibe un alta de producto con el texto crudo del proveedor (en sus propias unidades) y devuelve medidas y peso normalizados a **cm y kg**, validados contra un contrato Pydantic, con tokens, costo y latencia. Sale de la HU1 de la PE1.

1 Contrato de datos (src/contrato.py)

Entrada	
`sku_id`	patrón `^SKU-\d{4,8}$`
`descripcion`	3 a 200 caracteres
`subcategoria`	Enum: snacks, bebidas, almacen, carnes, lacteos, limpieza, perfumeria
`texto_proveedor`	5 a 2000 caracteres

Salida	
`estado`	Enum: ok · convertido · revisar · fuera_de_tema
`largo/alto/ancho_cm`	rango 0 < x ≤ 200 (o null)
`peso_kg`	rango 0 < x ≤ 50 (o null)
`unidades_origen`	lista de Enum (mm, cm, g, kg...), mín. 1
`confianza`	Enum: alta · media · baja
`motivo`	15 a 400 caracteres
validación cruzada	ok/convertido exigen las 4 medidas · fuera_de_tema sin medidas · revisar ≠ confianza alta

2 Ejecuciones reales (salida de la terminal)

Exitosa: "Caja 210 × 60 × 45 mm, peso neto 350 grs"

```
=== EJECUCIÓN OK ===
{
  "ok": true,
  "salida": {
    "sku_id": "SKU-10231",
    "estado": "convertido",
    "largo_cm": 21.0, "alto_cm": 6.0,
    "ancho_cm": 4.5, "peso_kg": 0.35,
    "unidades_origen": ["mm", "g"],
    "confianza": "alta",
    "motivo": "Conversión de mm a cm y g a kg
     realizada correctamente según las
     dimensiones y peso neto proporcionados."
  },
  "metricas": {
    "intentos": 1,
    "tokens_in": 321, "tokens_out": 141,
    "costo_usd": 0.00029175,
    "latencia_ms": 758
  }
}
```

Con error: SKU sin formato, descripción corta, subcategoría inexistente

```
=== EJECUCIÓN CON ERROR ===
{
  "ok": false,
  "error": {
    "codigo": "ENTRADA_INVALIDA",
    "campos": [
      "sku_id",
      "descripcion",
      "subcategoria"
    ]
  },
  "metricas": null
}

# Detalle por campo (Pydantic):
sku_id → String should match
  pattern '^SKU-\d{4,8}$'
descripcion → String should have at
  least 3 characters
subcategoria → Input should be 'snacks',
  'bebidas', 'almacen', 'carnes', ...
```

La entrada inválida se rechaza **antes** de llamar al modelo: no gasta tokens.

3 Medición: 5 consultas reales (01/11/2026)

#	Consulta	¿Validó?	Estado (esperado)	Tokens (entrada / salida)	Costo USD*	Latencia ms
1	Unidades del proveedor (mm y g)	Sí	convertido (convertido) ✓	347 / 138	0,000294	6.969
2	Datos ya en cm y kg	Sí	ok (ok) ✓	335 / 129	0,000277	7.966
3	Pack (6×70 g)	Sí	ok (convertido) ×	326 / 153	0,000311	9.493
4	Confusa (peso sin unidad)	Sí	convertido (revisar) ×	324 / 134	0,000282	830
5	Fuera de tema	Sí	fuera_de_tema (fuera_de_tema) ✓	320 / 119	0,000258	6.167
	Promedio	**100%**	**60% de acierto**	**330 / 135**	**0,000285**	**6.285**

* Costo hipotético con la tarifa paga publicada de gemini-3.1-flash-lite (USD 0,25 / 1,50 por millón de tokens de entrada / salida, incluido razonamiento). En el nivel gratuito usado el costo real fue USD 0. Tokens tomados de `usage_metadata`; latencia medida con `time.perf_counter()`.

Contra la meta de la PE1 (KPI técnico ≥95% de datos coherentes): el formato validó el 100% de las veces, pero el estado fue correcto solo en el 60% → **no cumple todavía** (muestra de 5, indicativa). Las dos fallas son semánticas, no de formato: el pack 6×70 g se marcó "ok" aunque hubo que convertir, y el caso confuso ("aprox 500") se dio por convertido en vez de mandarse a revisar. Esto confirma el riesgo R1 de la PE1 y justifica la aprobación humana. El costo (USD 0,0003 por consulta) es despreciable frente a los ~6 min manuales por SKU; la latencia (6,3 s promedio, con picos en el nivel gratuito) es aceptable para un proceso de altas en tandas, no en tiempo real.

4 Qué hace el sistema cuando falla la validación

Si la salida del modelo no valida, el motor registra qué campo falló (la ruta `loc` de Pydantic), le devuelve ese error al modelo y reintenta una sola vez; si vuelve a fallar, devuelve un error estructurado `SALIDA_INVALIDA` con la lista de campos y nada se carga en el catálogo.

- **Entrada inválida** → `ENTRADA_INVALIDA` con los campos, sin llamar al modelo.
- **Errores de la API** (límite de uso, sin conexión, clave inválida) → error con código y mensaje, en la misma forma `{ok, salida, error, metricas}`.
- Los tokens de ambos intentos se suman: un reintento también cuesta.

5 Estructura del repositorio (tag pe2)

```
fde-medidas-catalogo/          (tag: pe2)
├── README.md            # cómo correrlo desde un clon limpio
├── requirements.txt
├── .env.example         # nombres de variables, sin valores
├── .gitignore           # incluye .env
├── prompts/
│   ├── sistema.md       # prompt de sistema (plantilla)
│   ├── usuario.md       # plantilla de la consulta
│   └── correccion.md    # mensaje para el reintento
├── data/consultas.json  # las 5 consultas de prueba
├── resultados/mediciones.md
├── notebooks/pe2_colab.ipynb
├── tests/test_contrato.py   # 9 pruebas sin API key
└── src/
    ├── contrato.py      # Entrada / Salida (Pydantic)
    ├── proveedores.py   # Gemini · Anthropic · OpenAI · mock
    ├── motor.py         # motor + CLI
    └── medir.py         # corre las 5 consultas
```

Cómo se corre:

```
pip install -r requirements.txt
cp .env.example .env   # completar GEMINI_API_KEY
python -m src.motor --sku SKU-10231 \
  --descripcion "Galletitas 350 g" \
  --subcategoria snacks \
  --texto "Caja 210x60x45 mm, 350 grs"
python -m src.medir    # las 5 consultas
pytest -q              # pruebas sin API key
```

Extra: el proveedor está abstraído: `PROVEEDOR=anthropic` u `openai` cambia de modelo sin tocar el resto del código.

6 Decisiones y trade-offs

Modelo. `gemini-3.1-flash-lite`: la tarea es extraer y convertir números de un texto corto, no razonar en profundidad, así que un modelo chico alcanza en formato (100% válido) a un costo de USD 0,0003 por consulta y con nivel gratuito para el curso. La consigna menciona OpenAI o Anthropic; el motor ya soporta Anthropic (`claude-haiku`) cambiando una variable, y la comparación de calidad entre modelos queda para la PE6. Si el acierto no mejora con el prompt, el siguiente paso es probar un modelo más grande y medir si el aumento de costo lo justifica.

Prompt de sistema (`prompts/sistema.md`, separado del código). Le da un rol acotado (asistente de datos maestros), reglas explícitas de conversión (mm→cm, g→kg, packs = peso total), la orden de **no inventar** y mandar a "revisar" ante datos faltantes o ambiguos, el caso "fuera_de_tema", y la regla de que el texto del proveedor es un dato y no una instrucción (defensa contra inyección de prompt). Se usa temperatura 0 y salida JSON. Las dos fallas medidas muestran qué ajustar: reforzar que un pack siempre es "convertido" y que un peso sin unidad siempre es "revisar".

Cuando la salida no valida, el sistema prefiere frenar antes que cargar un dato dudoso: reintenta una vez con el error concreto y, si sigue mal, devuelve el error con los campos. Como el formato correcto no garantiza que el contenido sea verdadero (las fallas de la tabla pasaron la validación), toda propuesta pasa por la aprobación de Datamaestra antes de crear el ticket (HU2 de la PE1).